Computability – Topic 8

Yonghao Lee

March 2, 2026

Contents

1 Rice's Theorem **2**

2 Applying Rice's Theorem **2**
2.1 When Can You Use Rice's Theorem? . . . 3
2.2 Examples Where Rice's Theorem Applies . . . 3
2.3 Examples Where Rice's Theorem Does NOT Apply . . . 4
2.4 Quick-Reference Checklist . . . 4

3 Time Complexity **4**
3.1 Analyzing Algorithms . . . 5
3.2 Complexity Relationships Among Models . . . 5
3.2.1 Multi-Tape vs. Single-Tape . . . 5
3.2.2 Nondeterministic vs. Deterministic . . . 5
3.2.3 Summary of Simulation Costs . . . 6
3.3 The Class P . . . 6
3.3.1 Polynomial Time . . . 6
3.3.2 Examples of Problems in P . . . 7
3.4 The Class NP . . . 9
3.5 Examples of Problems in NP . . . 11
3.5.1 SUBSET-SUM . . . 13
3.6 The Class coNP . . . 13

1 Rice's Theorem

A **property** of Turing machines is a set P of TMs. If $M \in P$, we say that M satisfies property P.

A property is **non-trivial** if there exists a machine that satisfies it and also a machine that does not; that is, P is neither the set of all TMs nor the empty set.

A property P is called **semantic** if whenever two machines have the same language, they agree on the property. Formally, for any two TMs M_1, M_2 such that $L(M_1) = L(M_2)$, we have

$$M_1 \in P \iff M_2 \in P.$$

We define the language of TMs satisfying P as

$$L_P = \{\langle M \rangle \mid M \in P\}.$$

Rice's Theorem: For any non-trivial semantic property P, we have $L_P \notin R$.

Lemma

Let P be a non-trivial semantic property such that $T_\emptyset \notin P$ (where $T_\emptyset$ is a TM with $L(T_\emptyset) = \emptyset$). Then $A_{\mathrm{TM}} \leq_{\mathrm{m}} L_P$.

Proof. Since P is non-trivial and $T_\emptyset \notin P$, there exists some TM $T \in P$.

Construction: Define $f(\langle M, w \rangle) = \langle M' \rangle$, where M' behaves as follows on input $x \in \Sigma^*$:

1. Simulate M on w.
2. If M rejects, reject. If M accepts, simulate T on x and output whatever T outputs.

Correctness:

- Suppose $\langle M, w \rangle \in A_{\mathrm{TM}}$. Then M accepts w, so M' behaves exactly like T on every input x. Hence $L(M') = L(T)$. Since $T \in P$ and P is semantic, we have $M' \in P$, so $\langle M' \rangle \in L_P$. ✓
- Suppose $\langle M, w \rangle \notin A_{\mathrm{TM}}$. Then M does not accept w, so M' either rejects or loops on every input x (it never reaches step 2). Hence $L(M') = \emptyset = L(T_\emptyset)$. Since $T_\emptyset \notin P$ and P is semantic, we have $M' \notin P$, so $\langle M' \rangle \notin L_P$. ✓

Computability: A TM can compute f by hard-coding M, w, and T into the description of M'.

Therefore $A_{\mathrm{TM}} \leq_{\mathrm{m}} L_P$. □

Proof of Rice's Theorem

Proof. Let P be a non-trivial semantic property.

Case 1: $T_\emptyset \notin P$. By the lemma, $A_{\mathrm{TM}} \leq_{\mathrm{m}} L_P$. Since $A_{\mathrm{TM}} \notin R$, we conclude $L_P \notin R$.

Case 2: $T_\emptyset \in P$. Consider the complement property $\bar{P} = \{M \mid M \notin P\}$. Then:

- $\bar{P}$ is non-trivial (since P is non-trivial).
- $\bar{P}$ is semantic (since P is semantic).
- $T_\emptyset \notin \bar{P}$ (since $T_\emptyset \in P$).

So by the lemma applied to $\bar{P}$, we have $A_{\mathrm{TM}} \leq_{\mathrm{m}} L_{\bar{P}}$, hence $L_{\bar{P}} \notin R$. Since $L_{\bar{P}} = \overline{L_P}$ and R is closed under complement, we conclude $L_P \notin R$. □

2 Applying Rice's Theorem

Rice's theorem is a shortcut: it lets you instantly prove undecidability without constructing a full reduction, as long as you verify three conditions.

2.1 When Can You Use Rice's Theorem?

You can apply Rice's theorem to prove $L_P \notin R$ whenever the language has the form:

$$L = \{\langle M \rangle \mid \text{some condition on } L(M)\}$$

You need to check three things:

1. **Form:** Is the input just $\langle M \rangle$? (Not $\langle M, w \rangle$ or something else.)
2. **Semantic:** Does the condition depend only on $L(M)$, not on the internal structure of M?
3. **Non-trivial:** Can you find one TM that satisfies the property and one that does not?

If all three hold, the language is undecidable by Rice's theorem.

2.2 Examples Where Rice's Theorem Applies

Example 2.1: $\varepsilon \in L(M)$

Claim: $L = \{\langle M \rangle \mid \varepsilon \in L(M)\}$ is undecidable.
We verify the conditions of Rice's theorem:

- **Form:** The input is $\langle M \rangle$ alone. ✓
- **Semantic:** The condition "$\varepsilon \in L(M)$" depends only on what $L(M)$ is. ✓
- **Non-trivial:**
  - *Satisfies:* Let M_1 be a TM that accepts every input. Then $L(M_1) = \Sigma^*$, so $\varepsilon \in L(M_1)$.
  - *Does not satisfy:* $T_\emptyset$ (the TM that rejects everything). Then $L(T_\emptyset) = \emptyset$, so $\varepsilon \notin L(T_\emptyset)$.

  ✓

By Rice's theorem, $L \notin R$. □

Example 2.2: $L(M)$ **is finite**

Claim: $L = \{\langle M \rangle \mid L(M) \text{ is finite}\}$ is undecidable.
We verify the conditions:

- **Form:** Input is $\langle M \rangle$ alone. ✓
- **Semantic:** Finiteness is a property of the language $L(M)$. ✓
- **Non-trivial:**
  - *Satisfies:* $T_\emptyset$, since $L(T_\emptyset) = \emptyset$ is finite.
  - *Does not satisfy:* A TM that accepts everything, since $L(M) = \Sigma^*$ is infinite.

  ✓

By Rice's theorem, $L \notin R$. □

Example 2.3: $L(M) = \{0, 1\}$

Claim: $L = \{\langle M \rangle \mid L(M) = \{0, 1\}\}$ is undecidable.
We verify the conditions:

- **Form:** Input is $\langle M \rangle$ alone. ✓
- **Semantic:** The condition asks whether $L(M)$ equals a specific set. ✓
- **Non-trivial:**
  - *Satisfies:* Build a TM that accepts exactly $\{0, 1\}$.
  - *Does not satisfy:* $T_\emptyset$, since $\emptyset \neq \{0, 1\}$.

  ✓

By Rice's theorem, $L \notin R$. □

Example 2.4: $L(M)$ is a CFL

Claim: $L = \{\langle M \rangle \mid L(M) \text{ is a context-free language}\}$ is undecidable.
We verify the conditions:

- **Form:** Input is $\langle M \rangle$ alone. ✓
- **Semantic:** Whether $L(M)$ is context-free depends only on the language. ✓
- **Non-trivial:**
  - *Satisfies:* $T_\emptyset$, since $\emptyset$ is a CFL.
  - *Does not satisfy:* A TM that recognizes $\{a^n b^n c^n \mid n \geq 0\}$, which is not a CFL.

  ✓

By Rice's theorem, $L \notin R$. □

2.3 Examples Where Rice's Theorem Does NOT Apply

Example 2.5: Syntactic Property

$L = \{\langle M \rangle \mid M \text{ has at most 5 states}\}$
This is a property of the *machine itself*, not of its language. Two TMs can recognize the same language but have different numbers of states. The property is **not semantic**, so Rice's theorem does not apply.
In fact, this language *is* decidable: a decider can simply inspect the encoding $\langle M \rangle$ and count the states.

Example 2.6: Input is not just $\langle M \rangle$

$L = \{\langle M, w \rangle \mid M \text{ accepts } w\}$
The input is $\langle M, w \rangle$, not $\langle M \rangle$ alone. Rice's theorem does not apply here. This is A_{TM}, which is undecidable, but must be proved via diagonalization, not Rice.

Example 2.7: Trivial Property

$L = \{\langle M \rangle \mid L(M) \text{ is RE}\}$
Every TM recognizes an RE language by definition. So *every* TM satisfies this property—it is trivial. Rice's theorem does not apply.
Indeed, L is simply the set of all valid TM encodings, which is decidable.

2.4 Quick-Reference Checklist

When asked "Is L decidable?" for a language involving TMs, run through the following:

Check	**Question**
Form	Is $L = \{\langle M \rangle \mid \ldots\}$? (just a TM, no extra input)
Semantic	Does the condition depend only on $L(M)$, not on M's structure?
Non-trivial	Can you find one TM in P and one TM not in P?
All yes	$\Rightarrow$ **Undecidable by Rice's theorem**
Any no	$\Rightarrow$ Rice does not apply; use another method

3 Time Complexity

Even when a problem is decidable and thus computationally solvable in principle, it may not be solvable in practice if the solution requires an inordinate amount of time or memory. As a result, we need to consider *computational complexity* in terms of time, memory, or other resources required for solving computational problems. We begin with time.

3.1 Analyzing Algorithms

Some notations are needed for classifying languages according to their time requirements.

Definition 3.1: Time Complexity Class

Let $t : \mathbb{N} \to \mathbb{R}^+$ be a function. The **time complexity class** $\text{TIME}(t(n))$ is the collection of all languages that are decidable by an $O(t(n))$ time Turing machine.

Example 3.1: 0^k1^k Complexity

The language $A = \{0^k1^k \mid k \geq 0\}$ can be decided by a single-tape TM in time $O(n^2)$ (by repeatedly scanning and crossing off matching 0s and 1s). Therefore $A \in \text{TIME}(n^2)$.

3.2 Complexity Relationships Among Models

We want to examine how the choice of computational model affects the time complexity of languages. We consider three models: the single-tape TM, the multi-tape TM, and the nondeterministic TM.

3.2.1 Multi-Tape vs. Single-Tape

Theorem 3.1: Multi-Tape to Single-Tape Simulation

Let $t(n)$ be a function where $t(n) \geq n$. Then every $t(n)$ time multitape TM has an equivalent $O(t^2(n))$ time single-tape TM.

Proof intuition. A single-tape TM S simulates a k-tape TM M by storing the contents of all k tapes on a single tape, separated by delimiters, with dotted symbols to mark the head positions. Each step of M requires S to:

1. Scan the entire active portion of the tape to read the symbols under each virtual head — this takes $O(t(n))$ time since the tape contents grow by at most one symbol per step.
2. Make a second pass to update the tape contents and move the virtual heads.

Since M runs for $t(n)$ steps and each step takes $O(t(n))$ work to simulate, the total time is $O(t(n)) \cdot O(t(n)) = O(t^2(n))$.

3.2.2 Nondeterministic vs. Deterministic

Definition 3.2: Running Time of an NTM

Let N be a nondeterministic TM that is a decider. The **running time** of N is the function $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of steps that N uses on *any branch* of its computation on *any input* of length n.

Remark 3.1: NTM Running Time

This definition does not correspond to any real-world computing device. Rather, it is a useful mathematical abstraction that assists in characterizing the complexity of an important class of computational problems.

Theorem 3.2: NTM to Deterministic Simulation

Let $t(n)$ be a function where $t(n) \geq n$. Then every $t(n)$ time nondeterministic single-tape TM has an equivalent $2^{O(t(n))}$ time deterministic single-tape TM.

Proof intuition. A deterministic TM D simulates the NTM N by exploring all possible branches of N's computation tree using, e.g., breadth-first search:

- The computation tree of N on input w has depth at most $t(n)$ (since N runs in time $t(n)$).
- If N has at most b nondeterministic choices at each step, the tree has at most $b^{t(n)}$ leaves.
- D systematically tries every root-to-leaf path. Each path takes $O(t(n))$ steps to simulate.
- Total time: $O(t(n)) \cdot b^{t(n)} = 2^{O(t(n))}$.

3.2.3 Summary of Simulation Costs

Simulation	**Time Overhead**
Multi-tape TM $\to$ Single-tape TM	At most *polynomial* (quadratic): $O(t^2(n))$
NTM $\to$ Deterministic TM	At most *exponential*: $2^{O(t(n))}$

This distinction—polynomial vs. exponential overhead—is the key observation that motivates the definitions that follow.

3.3 The Class P

On one hand, we demonstrated at most a polynomial (quadratic) difference between the time complexity of problems on single-tape and multitape deterministic TMs. On the other hand, we showed at most an exponential difference between the time complexity of problems on deterministic and nondeterministic TMs.

3.3.1 Polynomial Time

For our purposes, **polynomial differences** in running time are considered to be *small*, whereas **exponential differences** are considered to be *large*.

Exponential time algorithms typically arise when we solve problems by exhaustively searching through a space of solutions, called **brute-force search**.

Definition 3.3: Polynomial Equivalence of Models

Two computational models are said to be **polynomially equivalent** if each one can simulate the other with only a polynomial increase in running time. That is, if model A can solve a problem in time $t(n)$, then model B can solve the same problem in time $t(n)^c$ for some constant c, and vice versa.

Key fact: All *reasonable* deterministic computational models are polynomially equivalent. This includes single-tape TMs, multi-tape TMs, random-access machines (RAMs), and others. What "simulation" means here is that one model can carry out every computation of another model step-by-step, with the total number of steps increasing by at most a polynomial factor.

This is why we can safely ignore polynomial differences: they do not depend on the particular model of computation we choose. From here on, we focus on aspects of time complexity theory that are unaffected by polynomial differences in running time.

Definition 3.4: The Class P

P is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine. Formally:

$$P = \bigcup_{k \in \mathbb{N}} \mathrm{TIME}(n^k)$$

The class P is central to complexity theory for two reasons:

1. **Mathematical robustness.** P is invariant for all models of computation that are polynomially equivalent to the deterministic single-tape TM. Since all reasonable deterministic models are polynomially equivalent, the class P is the same regardless of which model we use. A language that is in P on a multi-tape TM is also in P on a single-tape TM (the polynomial may change, but it remains polynomial).

2. **Practical solvability.** P roughly corresponds to the class of problems that are realistically solvable on a computer. While not every polynomial-time algorithm is fast in practice (e.g., $O(n^{100})$ is technically polynomial but impractical), problems *outside* P are generally considered intractable, and problems *inside* P tend to have reasonable running times.

3.3.2 Examples of Problems in P

When we present a polynomial-time algorithm, we give a high-level description of it without reference to the features of a particular computational model. Doing so avoids tedious details of tapes and head motions. We follow certain conventions when describing an algorithm so that we can analyze it for polynomiality.

Many computational problems we are going to discuss contain encodings of graphs. One reasonable encoding of a graph is a list of its nodes and edges. Another is the adjacency matrix, which we learned from previous courses. In any reasonable representation of graphs, the size of the representation is a polynomial in the number of nodes. Thus, if we analyze an algorithm and show that its running time is polynomial in the number of nodes, we know that it is polynomial in the size of the input.

The first problem concerns directed graphs. The PATH problem is to determine whether a directed path exists from s to t.

$$\text{PATH} = \{\langle G, s, t\rangle \mid G \text{ is a directed graph that has a directed path from } s \text{ to } t\}$$

Theorem 3.3: PATH $\in P$

The PATH problem is in the class P.

We prove this theorem by presenting a polynomial-time algorithm that decides PATH.

A brute-force algorithm for PATH proceeds by examining all potential paths in G and determining whether any is a directed path from s to t. A potential path is a sequence of nodes in G having a length of at most m, where m is the number of nodes in G. However, the number of such potential paths is roughly m^m, which is exponential.

To get a polynomial-time algorithm for PATH, we must do something that avoids this brute-force approach. We can use Breadth-First Search (BFS).

Proof. We construct a Turing machine M to decide PATH.

M = "On input $\langle G, s, t\rangle$, where G is a directed graph with nodes s and t:

1. Place a mark on node s.
2. Repeat the following until no additional nodes are marked:
3. Scan all the edges of G. If an edge (a, b) is found going from a marked node a to an unmarked node b, mark node b.
4. If t is marked, *accept*. Otherwise, *reject*."

Obviously, the loop in stages 2 and 3 runs at most m times because each time, except the last, it marks at least one additional node in G. So the total number of stages executed is at most $1 + m + 1$, giving a running time that is polynomial in the size of G. □

Let's turn to another example of a polynomial-time algorithm. Two numbers are relatively prime if 1 is the largest integer that evenly divides them both. For example, 10 and 21 are relatively prime, even though neither of them is a prime number by itself. In contrast, 10 and 22 are not relatively prime because both are divisible by 2.

$$\text{RELPRIME} = \{\langle x, y\rangle \mid x \text{ and } y \text{ are relatively prime}\}$$

We prove that this language is in P.

Theorem 3.4: RELPRIME $\in P$

The language RELPRIME is in the class P.

Proof. The idea is to use the Euclidean algorithm for computing the greatest common divisor (gcd). The gcd of natural numbers x and y, written $\gcd(x, y)$, is the largest integer that evenly divides both x and y. For example, $\gcd(18, 24) = 6$. Obviously, x and y are relatively prime if and only if $\gcd(x, y) = 1$.

We define a Turing machine E that computes the gcd:

$E =$ "On input $\langle x, y\rangle$, where x and y are natural numbers in binary:

1. Repeat until $y = 0$:
2. Assign $x = x \bmod y$.
3. Exchange x and y.
4. Output x."

The algorithm R solves RELPRIME using E as a subroutine:

$R =$ "On input $\langle x, y\rangle$, where x and y are natural numbers in binary:

1. Run E on $\langle x, y\rangle$.
2. If the result is 1, *accept*; otherwise, *reject*."

Clearly, if E runs in polynomial time, so does R. Hence, we only need to analyze E for time complexity.

To determine the time complexity of E, we first show that every execution of stage 2 cuts the value of x by at least half. After stage 2 is executed, $x < y$ because of the nature of the modulo function. After stage 3, $x > y$ because we swapped them. Thus, when stage 2 is subsequently executed, $x > y$.

Notice that when $x > y$, we have two cases for $x \bmod y$:

- If $y \leq x/2$, then $x \bmod y < y \leq x/2$.
- If $y > x/2$, then x goes into y exactly once, so $x \bmod y = x - y < x/2$.

In both cases, the value of x is reduced by at least half.

Since the values of x and y are reduced by half every other step, the loop in E executes at most $2\min(\log_2 x, \log_2 y)$ times. Because the input numbers are represented in binary, their sizes (lengths) are approximately $\log_2 x$ and $\log_2 y$. Therefore, the number of loop iterations is linear in the size of the input. Because the modulo operation and swapping can be performed in polynomial time relative to the bit lengths, E runs in polynomial time. Consequently, R runs in polynomial time, proving RELPRIME $\in P$. □

Next we want to prove that every context-free language (CFL) is a member of P. We have already proved earlier that every CFL is decidable. The formal proof of polynomiality is omitted, but the idea goes like this:

Theorem 3.5: Every CFL is in P

Every context-free language is a member of P.

Proof Idea. Let L be a context-free language and G be a context-free grammar that generates L. We want to construct a polynomial-time algorithm that decides if an input string w of length n is in L.

First, we convert G into an equivalent grammar in **Chomsky Normal Form (CNF)**. We know that if a grammar is in CNF, any derivation of a string w of length $n \geq 1$ takes exactly $2n - 1$ steps.

A naive brute-force algorithm would try all possible derivations of length $2n - 1$ to see if any of them generate w. However, the number of such derivations branches at each step, leading to a number of possibilities that grows exponentially with n. Therefore, this brute-force approach is not a polynomial-time solution.

To achieve a polynomial-time algorithm, we use a **dynamic programming** approach known as the **CYK (Cocke–Younger–Kasami) algorithm**. The algorithm avoids redundant work by building up solutions for smaller substrings first.

The algorithm proceeds as follows:

1. Create an $n \times n$ table where the entry at (i, j) stores the set of all variables in G that can generate the substring of w starting at position i and having length j.
2. Fill the table starting with the shortest substrings of length 1. For each character in w, find which variables generate it directly (using rules of the form $A \to a$).
3. Iteratively fill the table for substrings of length 2, 3, and so on, up to length n. To find out which variables can generate a substring of length j, the algorithm looks at all possible ways to split the substring into two shorter parts. It then checks the already-filled table entries for those parts to see if any rule $A \to BC$ can combine them.
4. If the start variable S is in the table entry for the full string (starting at position 1 with length n), *accept*; otherwise, *reject*.

Time Complexity Analysis: The table contains roughly $n^2/2$ cells. To compute the value of each cell, the algorithm checks at most n possible splits. Checking each split takes constant time relative to the size of the grammar (which is a constant independent of n). Therefore, the total number of operations is bounded by $O(n^2 \cdot n) = O(n^3)$.

Because $O(n^3)$ is a polynomial, this algorithm runs in polynomial time. Hence, every CFL is in P. $\square$

3.4 The Class NP

Attempts to avoid brute force in certain other problems, including many interesting and useful ones, haven't been successful, and polynomial-time algorithms that solve them aren't known to exist. One remarkable discovery concerning the question of solving these problems is that a polynomial-time algorithm for one such problem can be used to solve an entire class of problems.

A Hamiltonian path in a directed graph G is a directed path that goes through each node exactly once. Let

$$\text{HAMPATH} = \{\langle G, s, t\rangle \mid G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t\}$$

We could use a brute-force algorithm to solve it in exponential time, and we do not know whether HAMPATH is solvable in polynomial time.

The HAMPATH problem has a feature called *polynomial verifiability* that is important for understanding its complexity. Even though we do not know of a fast way to determine whether a graph contains a Hamiltonian path, if such a path were discovered somehow, we could easily convince someone else of its existence simply by presenting it. In other words, verifying the existence of a Hamiltonian path may be much easier than determining its existence.

Another polynomially verifiable problem is compositeness. Recall that a natural number is composite if it is the product of two integers greater than 1:

$$\text{COMPOSITES} = \{x \mid x = pq, \text{ for integers } p, q > 1\}$$

We can easily verify that a number is composite—all that is needed is a divisor of that number.

Some problems may not be polynomially verifiable. For example, take $\overline{\text{HAMPATH}}$, the complement of the HAMPATH problem. Even if we could somehow determine that a graph did not have a Hamiltonian path, we don't know of a way for someone else to verify its nonexistence without using the same exponential-time algorithm for making the determination in the first place.

Explanation: To prove a path *exists*, you just need to show one valid example (a short proof). To prove a path *does not exist*, you essentially have to prove that all $n!$ possible paths fail. We currently do not have a compact, polynomial-length "certificate of nonexistence" that a verifier can check quickly.

Now we define:

Definition 3.5: Verifier

A verifier for a language A is an algorithm V, where

$$A = \{w \mid V \text{ accepts } \langle w, c\rangle \text{ for some string } c\}$$

We measure the time of a verifier only in terms of the length of w, so a polynomial-time verifier runs in polynomial time in the length of w. A language A is polynomially verifiable if it has a polynomial-time verifier.

A verifier uses additional information, represented by the symbol c in the definition, to verify that a string w is a member of A. This information is called a *certificate* or *proof* of membership in A. Observe that for polynomial verifiers, the certificate has polynomial length in the length of w, because that is all the verifier can access in its time bound.

Explanation: A polynomial-time verifier is bounded to run in $O(n^k)$ steps. In that amount of time, the Turing Machine's reading head can physically only read at most $O(n^k)$ symbols of the certificate. Any part of the certificate longer than that is completely unread and useless. Therefore, we can safely assume the useful certificate is at most polynomial in length.

For the HAMPATH problem, a certificate for a string $\langle G, s, t\rangle \in$ HAMPATH simply is a Hamiltonian path from s to t. For the COMPOSITES problem, a certificate for the composite number x simply is one of its divisors. In both cases, the verifier can check in polynomial time that the input is in the language when it is given the certificate.

Therefore, we define:

Definition 3.6: NP

NP is the class of languages that have polynomial-time verifiers.

The class NP is important because it contains many problems of practical interest. The term NP comes from *nondeterministic polynomial time* and is derived from an alternative characterization by using nondeterministic polynomial-time Turing machines.

The following is an NTM that decides the HAMPATH problem, therefore it is an alternative definition of NP. Recall that we defined the time of an NTM to be the time used by the longest computation branch.

$N_1 =$ On input $\langle G, s, t\rangle$, where G is a directed graph with nodes s and t:

1. Write a list of m numbers $p_1, \ldots, p_m$, where m is the number of nodes in G. Each number in the list is nondeterministically selected to be between 1 and m.

   (**Explanation:** The NTM essentially "guesses" a path. Nondeterminism means the machine branches into many parallel computation paths; in each branch, it writes down a different sequence of m nodes. At least one of these branches will guess the correct Hamiltonian path, if it exists.)

2. Check for repetitions in the list. If any are found, reject.

3. Check whether $s = p_1$ and $t = p_m$. If either fails, reject.

4. For each i between 1 and $m - 1$, check whether (p_i, p_{i+1}) is an edge of G. If not, reject. Otherwise, all tests have been passed, so accept.

Algorithm Explanation: This algorithm branches to guess every possible sequence of m vertices. On each branch, it verifies if that specific guess is a valid Hamiltonian path. It checks if every node is distinct (Step 2), if it starts and ends at the right places (Step 3), and if the nodes are actually connected by edges in the graph (Step 4). If any branch survives all these checks, the NTM accepts.

In stage 1, the nondeterministic selection clearly runs in polynomial time.

Why? Writing down m numbers (where m is the number of vertices, so $m \leq n$) takes time proportional to m. Even if it takes $\log m$ bits to write each number, the total time to write the list is $O(m \log m)$, which is strictly polynomial relative to the input size.

In stages 2 and 3, each part is a simple check, so together they take polynomial time. Finally, stage 4 also clearly runs in polynomial time. Thus, this algorithm runs in nondeterministic polynomial time.

Theorem 3.6: The Class NP

A language is in NP if and only if it is decided by some nondeterministic polynomial-time Turing machine.

Proof. We show how to convert a polynomial-time verifier to an equivalent polynomial-time NTM and vice versa.

Forward Direction: Let $A \in \text{NP}$, and show that A is decided by a polynomial-time NTM N. Let V be the polynomial-time verifier for A that exists by the definition of NP. Assume that V is a TM that runs in time n^k and construct N as follows:

$N =$ On input w of length n:

1. Nondeterministically select string c of length at most n^k.
2. Run V on $\langle w, c \rangle$.
3. If V accepts, accept; otherwise, reject.

Reverse Direction: To prove the other direction of the theorem, assume that A is decided by a polynomial-time NTM N and construct a polynomial-time deterministic verifier V as follows:

$V =$ On input $\langle w, c \rangle$ where w and c are strings:

1. Simulate N on input w, treating each symbol of c as a description of the nondeterministic choice to make at each step.
2. If this branch of N's computation accepts, accept; otherwise, reject.

Explanation of this proof step: (Isn't this just deterministic?) Yes, exactly! The verifier V must be a deterministic machine. An NTM works like a maze explorer that magically splits itself at every fork in the road. The certificate c acts as a "cheat sheet" or "map" (e.g., "turn left, then right, then left"). V is simply a normal, deterministic machine that walks through the maze following the exact turns written in c. If c is a valid map to the exit (the accept state), V accepts. Because the NTM N reaches the end in polynomial steps, the map c is only polynomial in length, and the deterministic walk V takes polynomial time. □

Then we define the class NTIME:

Definition 3.7: NTIME

$$\text{NTIME}(t(n)) = \{L \mid L \text{ is a language decided by an } O(t(n)) \text{ time nondeterministic TM}\}$$

As a result, $\text{NP} = \bigcup_k \text{NTIME}(n^k)$.

The class NP is insensitive to the choice of reasonable nondeterministic computational model because all such models are polynomially equivalent.

Explanation: Whether you define your NTM as having one tape, multiple tapes, or even model it as a Nondeterministic Random Access Machine (RAM), it doesn't matter. Any of these "reasonable" models can simulate any other reasonable model with at most a polynomial slowdown (for example, taking $O(t^2)$ instead of $O(t)$). Since polynomials raised to a power are still polynomials, the broad bucket of "polynomial time" (and thus the class NP) remains exactly the same regardless of the hardware model we use.

3.5 Examples of Problems in NP

A **clique** in an undirected graph is a subgraph wherein every two nodes are connected by an edge. A k-clique is a clique that contains exactly k nodes.

The clique problem is to determine whether a graph contains a clique of a specified size. Let:

$$\text{CLIQUE} = \{\langle G, k\rangle \mid G \text{ is an undirected graph with a } k\text{-clique}\}$$

Theorem 3.7: CLIQUE is in NP

The language CLIQUE is in the class NP.

Proof. We can prove this in two equivalent ways: by constructing a polynomial-time verifier, or by constructing a polynomial-time NTM.

Method 1: Constructing a Polynomial-Time Verifier

For a given graph G and an integer k, the certificate c is simply the subset of k nodes that supposedly form the clique.

We define the verifier V as follows:

V = On input $\langle\langle G, k\rangle, c\rangle$:

1. Test whether c is a set containing exactly k nodes from G.
2. Test whether G contains an edge connecting every possible pair of nodes in c.
3. If both tests pass, *accept*; otherwise, *reject*.

Complexity Analysis of V: Let n be the number of nodes in G and m be the number of edges.

- Step 1: Counting the nodes in c and verifying they belong to G takes $O(n)$ or $O(k \log n)$ time, which is polynomial.
- Step 2: The number of pairs of nodes in c is $\binom{k}{2} = \frac{k(k-1)}{2} = O(k^2)$. For each pair, checking if the edge exists in G takes at most $O(m)$ time (or $O(1)$ if using an adjacency matrix). Therefore, checking all edges takes at most $O(k^2 \cdot m)$ time.

Since both steps take time bounded by a polynomial in the size of the input graph, V is a polynomial-time verifier. Thus, CLIQUE $\in$ NP.

Method 2: Constructing a Nondeterministic Turing Machine (NTM)

Alternatively, we can design an NTM N that decides CLIQUE by "guessing" the k nodes.

N = On input $\langle G, k\rangle$:

1. Nondeterministically select a subset c of exactly k nodes from G.

   (**Explanation:** The NTM branches into many parallel computation paths. In each branch, it selects a different combination of k nodes from the graph.)
2. Test whether G contains all edges connecting the nodes in c.
3. If all pairs are connected, *accept*; otherwise, *reject*.

Complexity Analysis of N:

- Step 1: Nondeterministically selecting k nodes can be done by scanning through the list of nodes in G and nondeterministically choosing whether to include each node or not, keeping a counter to ensure exactly k are chosen. This takes $O(n)$ time on each branch.
- Step 2: Just like the verifier, checking all $\binom{k}{2}$ pairs for edges takes $O(k^2 \cdot m)$ deterministic time.

Because the maximum time taken on *any single computation branch* is $O(n + k^2 m)$, which is polynomial relative to the input size, N runs in nondeterministic polynomial time. Thus, CLIQUE $\in$ NP. $\square$

3.5.1 SUBSET-SUM

Now consider the SUBSET-SUM problem, which concerns integer arithmetic. We are given a multiset (a collection where repetition is allowed) of integers $S = \{x_1, x_2, \ldots, x_k\}$ and a target integer t. We want to determine whether there exists a subcollection of S whose elements sum to exactly t.

Formally:

$$\text{SUBSET-SUM} = \left\{ \langle S, t \rangle \;\middle|\; \begin{array}{l} S = \{x_1, \ldots, x_k\} \text{ is a multiset of integers and} \\ \exists\, \{y_1, \ldots, y_m\} \subseteq S \text{ such that } \displaystyle\sum_{i=1}^{m} y_i = t \end{array} \right\}$$

Theorem 3.8: SUBSET-SUM is in NP

The language SUBSET-SUM is in the class NP.

Proof. We prove membership in NP by presenting both a polynomial-time verifier and a polynomial-time NTM.

Method 1: Constructing a Polynomial-Time Verifier

The certificate c is a subcollection of numbers from S that supposedly sums to t.

V = On input $\langle \langle S, t \rangle, c \rangle$:

1. Test whether c is a subcollection of S (i.e., every element of c appears in S with sufficient multiplicity).
2. Compute the sum of all elements in c and test whether this sum equals t.
3. If both tests pass, *accept*; otherwise, *reject*.

Complexity Analysis: Step 1 requires comparing elements of c against those of S, which takes at most $O(k^2)$ time (where $k = |S|$). Step 2 requires summing at most k numbers, each of which has bit-length at most polynomial in the input size, so it also takes polynomial time. Therefore V is a polynomial-time verifier.

Method 2: Constructing a Nondeterministic Turing Machine

N = On input $\langle S, t \rangle$:

1. Nondeterministically select a subcollection c of elements from S (for each element, nondeterministically choose whether to include it or not).
2. Compute the sum of all elements in c and test whether this sum equals t.
3. If the sum equals t, *accept*; otherwise, *reject*.

Complexity Analysis: Step 1 takes $O(k)$ nondeterministic time. Step 2 takes polynomial time as before. Therefore N runs in nondeterministic polynomial time.

By either characterization, SUBSET-SUM $\in$ NP. □

3.6 The Class coNP

Sometimes, verifying that something is *not* present seems harder than verifying that it *is* present. For example, verifying that a graph *has* a Hamiltonian path is easy given the path as a certificate. But verifying that a graph *does not* have a Hamiltonian path appears to be much harder—we do not know of any short certificate that would allow a verifier to quickly confirm the absence of such a path.

This asymmetry motivates the following definition:

Definition 3.8: coNP

The class **coNP** is the set of languages whose *complements* are in NP. That is:

$$\text{coNP} = \{L \mid \overline{L} \in \text{NP}\}$$

Remark 3.2: P, NP, and coNP

We know the following relationships:

- $P \subseteq \text{NP}$ and $P \subseteq \text{coNP}$, since any polynomial-time decider can serve as both a verifier for membership and (by flipping the answer) a verifier for non-membership.
- It is an open question whether NP = coNP. Most researchers believe they are different, but no proof exists either way.
- If NP $\neq$ coNP, then P $\neq$ NP (since P is closed under complement).

For example, $\overline{\text{HAMPATH}} \in \text{coNP}$ by definition, since $\text{HAMPATH} \in \text{NP}$. However, we do not know whether $\overline{\text{HAMPATH}} \in \text{NP}$.